

## *Caveats of Parallelism (II)*

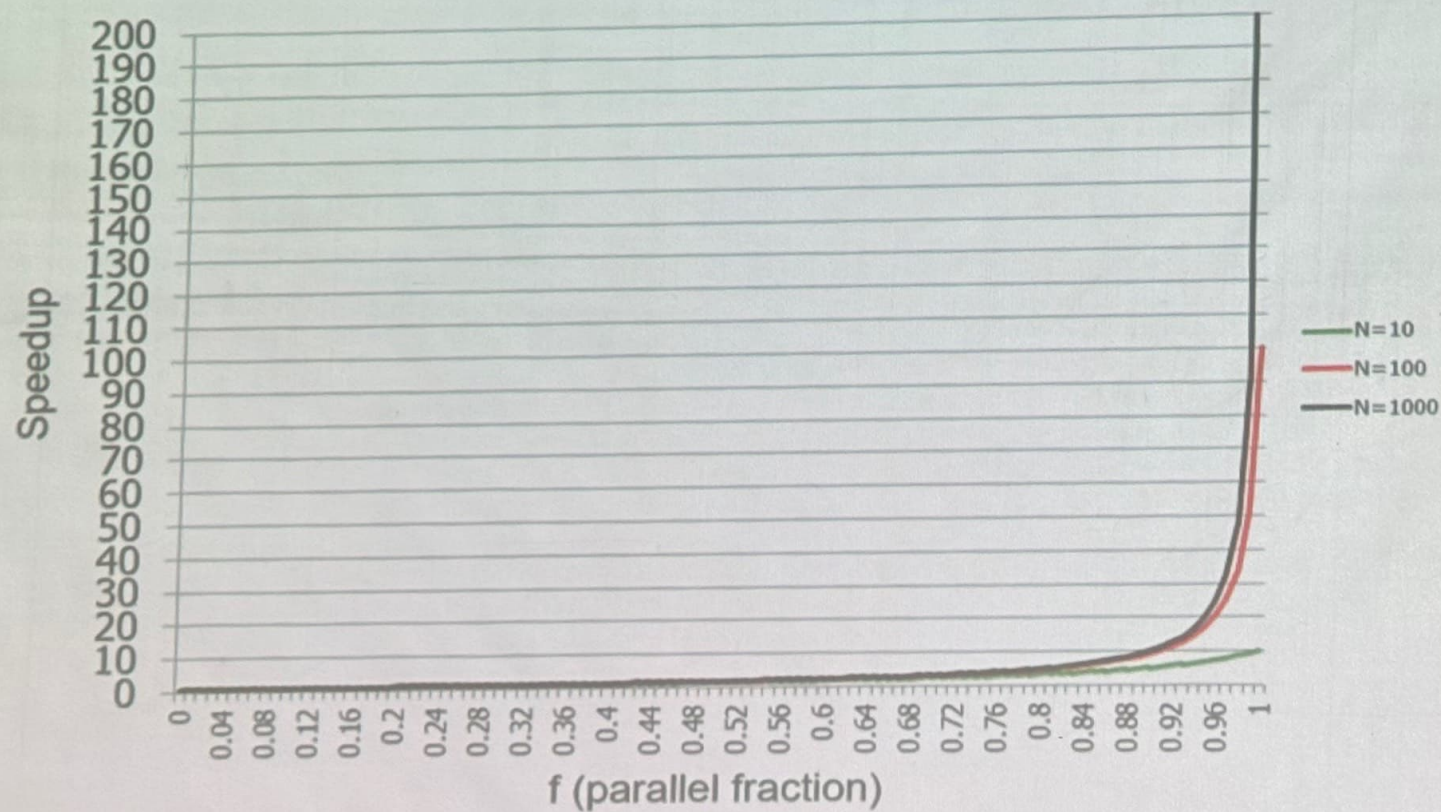
- Amdahl's Law

- f: Parallelizable fraction of a program
- N: Number of processors

$$\text{Speedup} = \frac{1}{1 - f + \frac{f}{N}}$$

- Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," AFIPS 1967.
- **Maximum speedup limited by serial portion: Serial bottleneck**
- **Parallel portion is usually not perfectly parallel**
  - Synchronization overhead (e.g., updates to shared data)
  - Load imbalance overhead (imperfect parallelization)
  - Resource sharing overhead (contention among N processors)

# Sequential Bottleneck





## What is the Maximum Speedup?

- $f$  = fraction of computation (algorithm) that is serial and cannot be parallelized
  - Data setup
  - Reading/writing to a single disk file

$$\bullet t_s = ft_s + (1-f) t_s$$

= serial portion + parallelizable portion

$$\bullet t_p = ft_s + ((1-f) t_s)/n$$

$$\bullet S(n) = t_s / (ft_s + ((1-f) t_s)/n)$$
$$= n / (1 + (n-1)f) \quad \leftarrow \text{Amdahl's Law}$$

$$\text{Limit as } n \rightarrow \infty = 1/f$$

## *Example of Amdahl's Law*

- Suppose that a calculation has a 4% serial portion, what is the limit of speedup on 16 processors?
  - $16 / (1 + (16 - 1) * .04) = 10$
  - What is the maximum speedup?
    - $1 / 0.04 = 25$



## *More to think about ...*

- Amdahl's law works on a *fixed* problem size
  - This is reasonable if your only goal is to solve a problem faster.
  - What if you also want to solve a larger problem?
    - Gustafson's Law (Scaled Speedup)

## *Gustafson's Law*

- Fix execution of on a single processor as
  - $s + p = \text{serial part} + \text{parallelizable part} = 1$
- $S(n) = (s + p)/(s + p/n)$   
 $= 1/(s + (1 - s)/n) = \text{Amdahl's law}$
- Now let,  $1 = s + \pi = \text{execution time on a parallel computer, with } \pi = \text{parallel part.}$ 
  - $S_s(n) = (s + \pi n)/(s + \pi) = n + (1-n)s$



## *More on Gustafson's Law*

- Derived by fixing the parallel execution time
  - Amdahl fixed the problem size
    - fixed serial execution time
  - For many practical situations, Gustafson's law makes more sense
    - Have a bigger computer, solve a bigger problem.
- Amdahl's law turns out to be too conservative for high-performance computing.

$$\begin{array}{c}
 P = 64 \\
 \alpha = 0.05 \quad | \quad 5\% \\
 \hline
 S_A \qquad \qquad S_G
 \end{array}$$

$$\begin{array}{c}
 P = 8 \\
 \alpha = 20\% \\
 \\
 S_A \qquad \qquad S_G
 \end{array}$$



$$P = 64$$

$$\alpha = 0.05 \mid 5\%$$

$$S_A \sim 15$$

$$S_G \sim 60$$

$$\boxed{8, 16, 32, 64}$$

$$P = 8$$

$$\alpha = 20\%$$

$$S_A \sim 3.3$$

$$S_G \sim 6.7$$

## *Linear and Superlinear Speedup*

- Linear speedup =  $N$ , for  $N$  processors
  - Parallel program is perfectly scalable
  - Rarely achieved in practice
- Superlinear Speedup
  - $S(N) > N$  for  $N$  processors
    - Theoretically not possible
    - Is this achievable on real machines?
      - Think about physical resources of  $N$  processors



# Efficiency

- $E(n) = S(n)/n * 100\%$
- A program with linear speedup is 100% efficient.

## Example questions

- Given a (scaled) speed up of 20 on 32 processors, what is the serial fraction from Amdahl's law?, From Gustafson's Law?
- A program attains 89% efficiency with a serial fraction of 2%. Approximately how many processors are being used according to Amdahl's law?



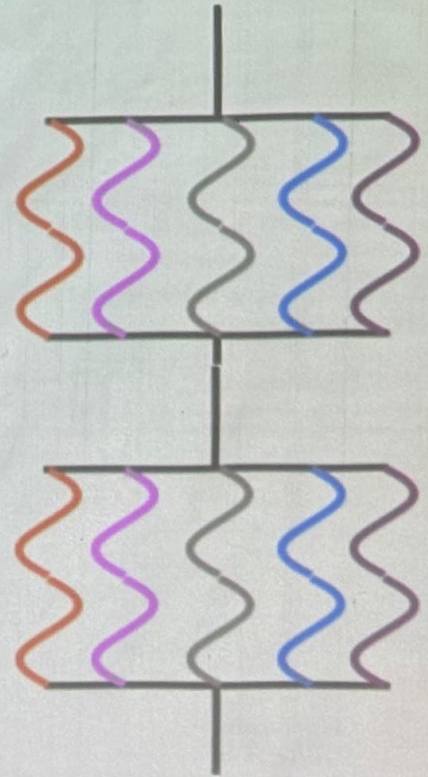
# Why the Sequential Bottleneck?

- Parallel machines have the sequential bottleneck
- Main cause: **Non-parallelizable operations on data** (e.g. non-parallelizable loops)

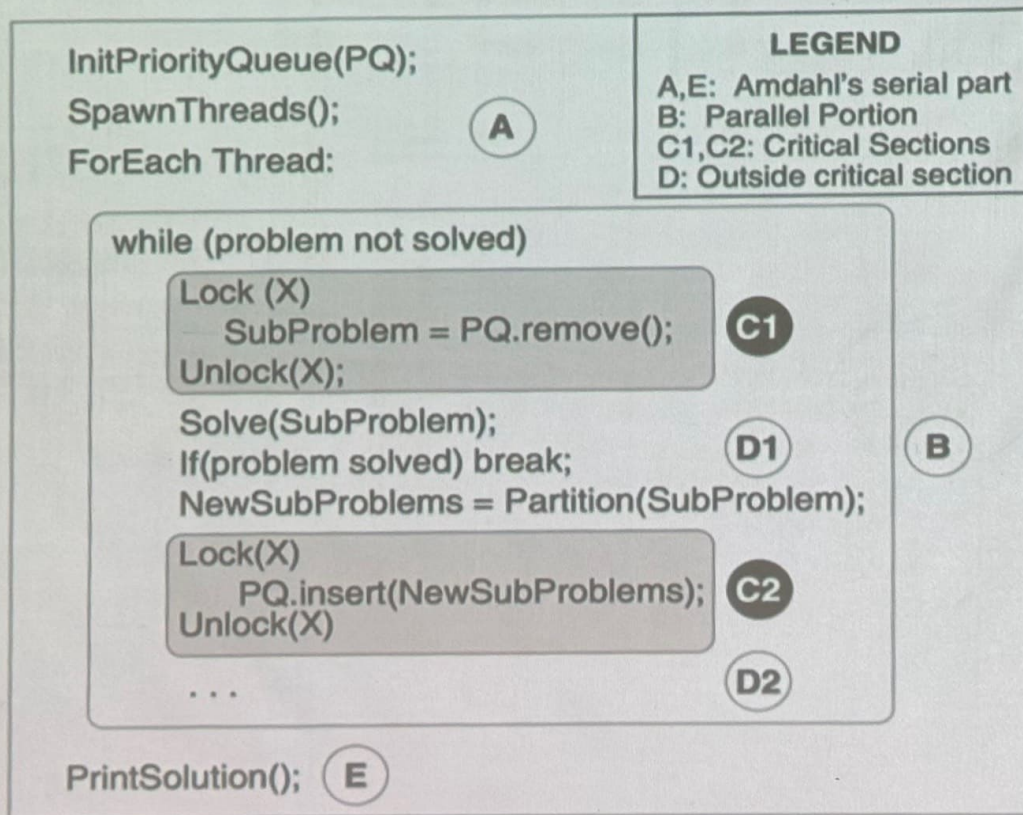
```
for ( i = 0 ; i < N; i++)
```

```
    A[i] = (A[i] + A[i-1]) / 2
```

- There are other causes as well:
  - Single thread prepares data and spawns parallel tasks (usually sequential)

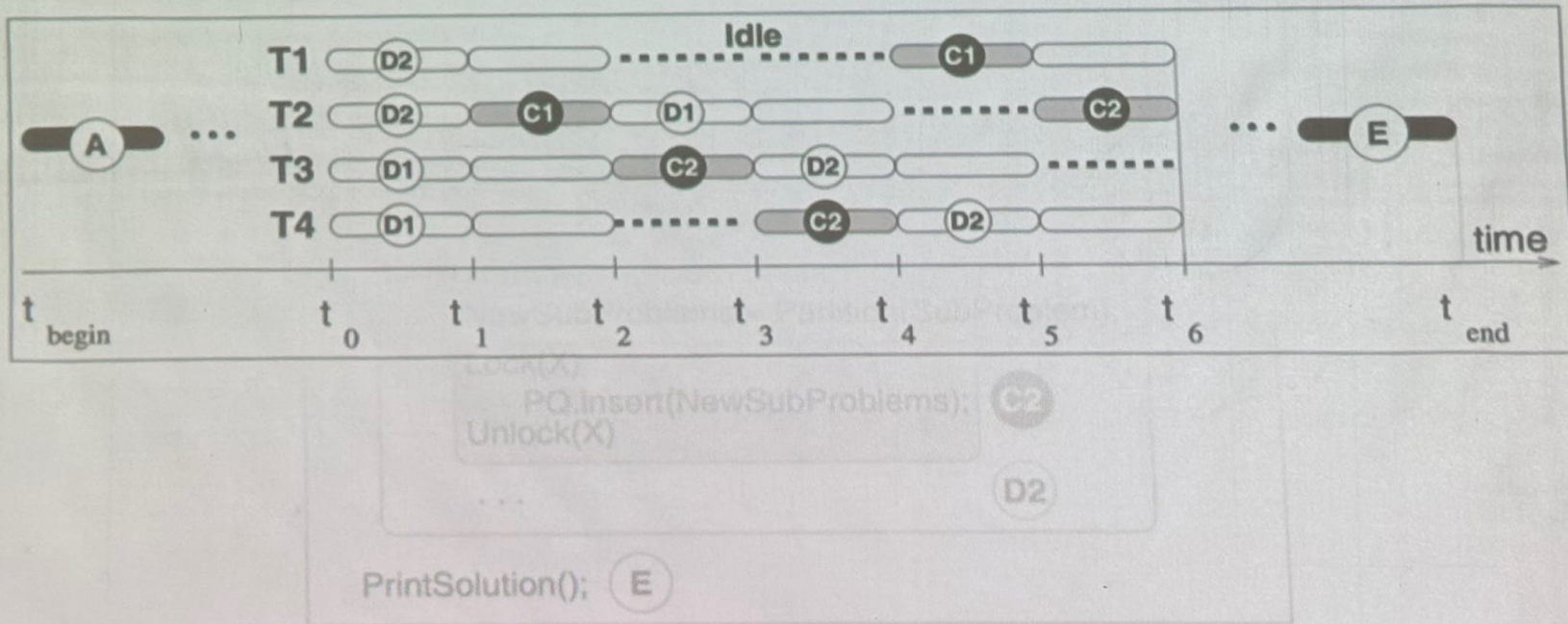


## Another Example of Sequential Bottleneck (I)





## Another Example of Sequential Bottleneck (II)





# ***Bottlenecks in Parallel Portion***

- **Synchronization:** Operations manipulating shared data cannot be parallelized
  - Locks, mutual exclusion, barrier synchronization
  - **Communication:** Tasks may need values from each other
    - Causes thread serialization when shared data is contended
- **Load Imbalance:** Parallel tasks may have different lengths
  - Due to imperfect parallelization or microarchitectural effects
  - Reduces speedup in parallel portion
- **Resource Contention:** Parallel tasks can share hardware resources, delaying each other
  - Replicating all resources (e.g., memory) expensive
  - Additional latency not present when each task runs alone



## ***Bottlenecks in Parallel Portion: Another view***

- Threads in a multi-threaded application can be inter-dependent
  - As opposed to threads from different applications
- Such threads can synchronize with each other
  - Locks, barriers, pipeline stages, condition variables, semaphores, ...
- Some threads can be on the critical path of execution due to synchronization; some threads are not



## **Bottlenecks in Parallel**

- Threads in a multi-threaded application can be inter-dependent
  - As opposed to threads from different applications
- Such threads can synchronize with each other
  - Locks, barriers, pipeline stages, condition variables, semaphores, ...
- Some threads can be on the critical path of execution due to synchronization; some threads are not
- Even within a thread, some “code segments” may be on the critical path of execution; some are not

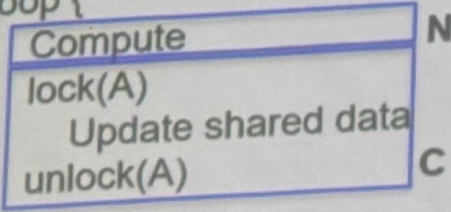


## Remember: Critical Section

- Enforce mutually exclusive access to shared data
- Only one thread can be executing it at a time

Each thread:

```
loop {  
    Compute  
    lock(A)  
    Update shared data  
    unlock(A)  
}
```



The diagram shows a code block for a thread loop. The code is: `loop {`, `Compute`, `lock(A)`, `Update shared data`, `unlock(A)`, `}`. A blue rectangular box highlights the `lock(A)`, `Update shared data`, and `unlock(A)` lines. To the right of the box, the letter 'N' is aligned with the top line and the letter 'C' is aligned with the bottom line, indicating the scope of the critical section.

# Remember. Barriers

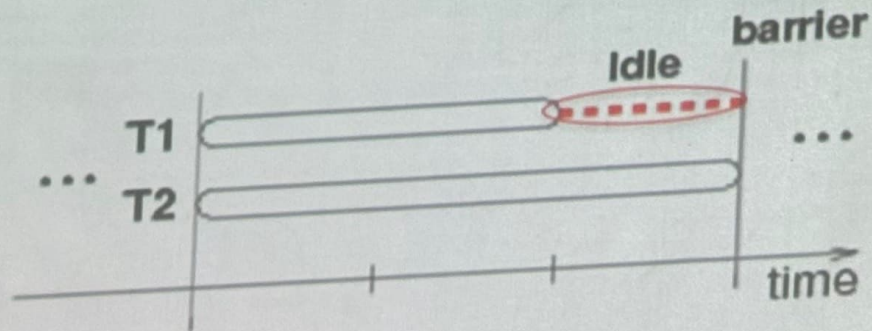
- Synchronization point
  - Threads have to wait until all threads reach the barrier
- Last thread arriving to the barrier is on the critical path

Each thread:

```
loop1 {  
  Compute
```

```
}  
barrier
```

```
loop2 {  
  Compute  
}
```

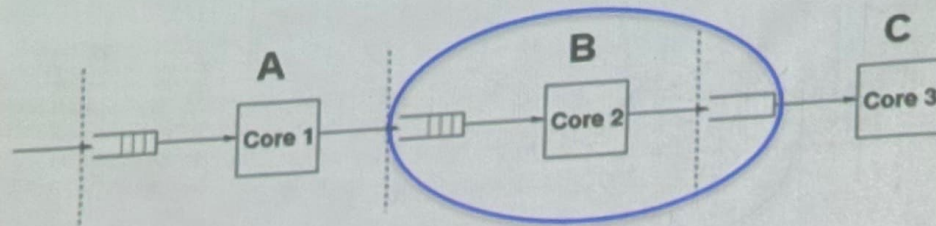




# Remember: Stages of Pipeline

- Loop iterations are statically divided into code segments called stages
- Threads execute stages on different cores
- Thread executing the slowest stage is on the critical path

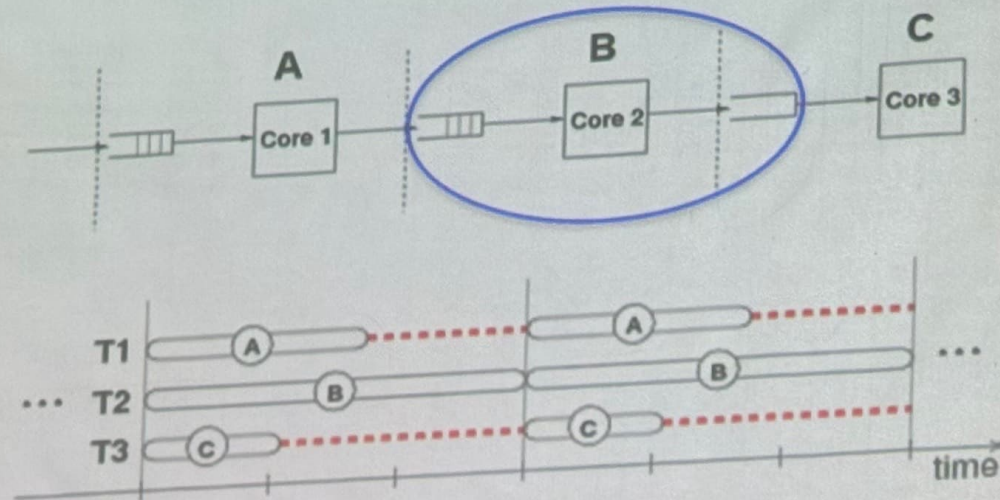
```
loop {  
  Compute1 A  
  Compute2 B  
  Compute3 C  
}
```



# Remember: Stages of Pipeline

- Loop iterations are statically divided into code segments called stages
- Threads execute stages on different cores
- Thread executing the slowest stage is on the critical path

```
loop {  
  Compute1 A  
  Compute2 B  
  Compute3 C  
}
```

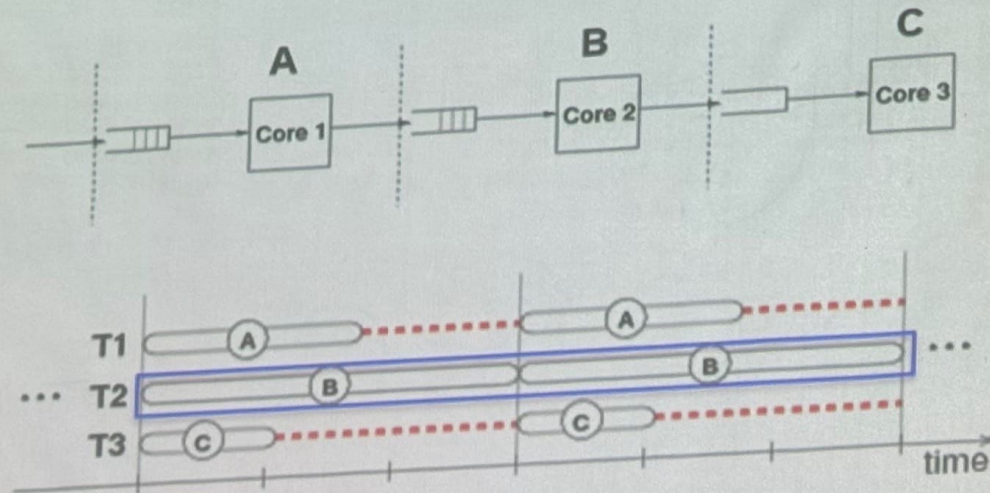




# Remember: Stages of Pipeline

- Loop iterations are statically divided into code segments called stages
- Threads execute stages on different cores
- Thread executing the slowest stage is on the critical path

```
loop {  
  Compute1 A  
  Compute2 B  
  Compute3 C  
}
```



# *Difficulty in Parallel Regions*

- Little difficulty if parallelism is natural
  - “Embarrassingly parallel” applications
  - Multimedia, physical simulation, graphics
  - Large web servers, databases?

Difficulty is in

- Getting parallel programs to work correctly
- Optimizing performance in the presence of bottlenecks



# Caching in Multiprocessors

- Caching not only complicates ordering of **all operations...**
  - A memory location can be present in multiple caches
  - Prevents the effect of a store or load to be seen by other processors → **makes it difficult for all processors to see the same global order of (all) memory operations**
- ... but it also complicates ordering of **operations on a single memory location**
  - A single memory location can be present in multiple caches
  - **Makes it difficult for processors that have cached the same location to have the correct value of that location (in the presence of updates to that location)**

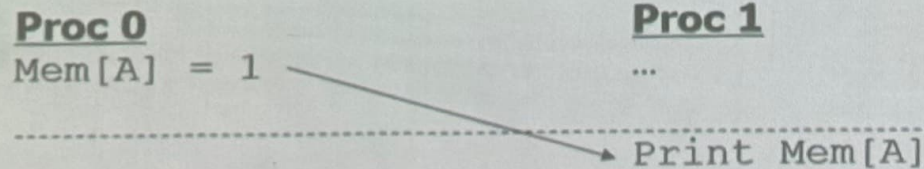
# Memory Consistency vs. Cache Coherence

- Consistency is about ordering of **all memory operations** from different processors (i.e., to different memory locations)
  - **Global ordering** of accesses to *all memory locations*
- Coherence is about ordering of **operations** from different processors **to the same memory location**
  - **Local ordering** of accesses to *each cache block*



# Shared Memory Model

- Many parallel programs communicate through shared memory
- Proc 0 writes to an address, followed by Proc 1 reading
  - This implies communication between the two





Emb. //

for  $(i=1:N)$

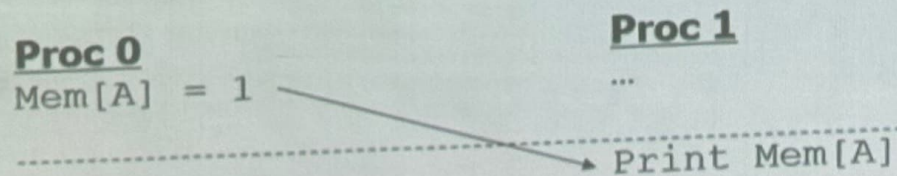
$a(i) = 2a(i)$

Shared Variables



# Shared Memory Model

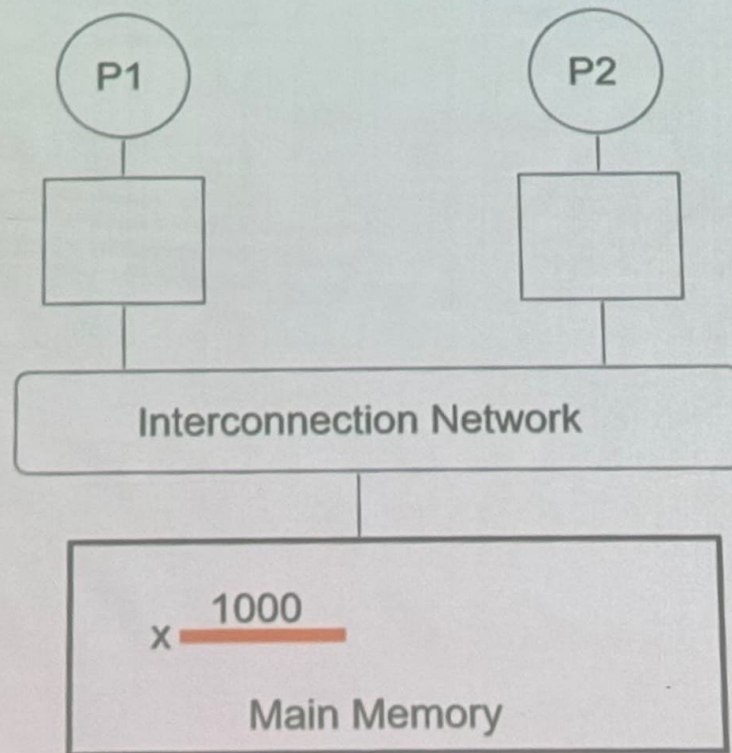
- Many parallel programs communicate through shared memory
- Proc 0 writes to an address, followed by Proc 1 reading
  - This implies communication between the two



- Each read should receive the value last written by anyone
  - This requires synchronization (what does last written mean?)
- What if Mem[A] is cached (at either end)?

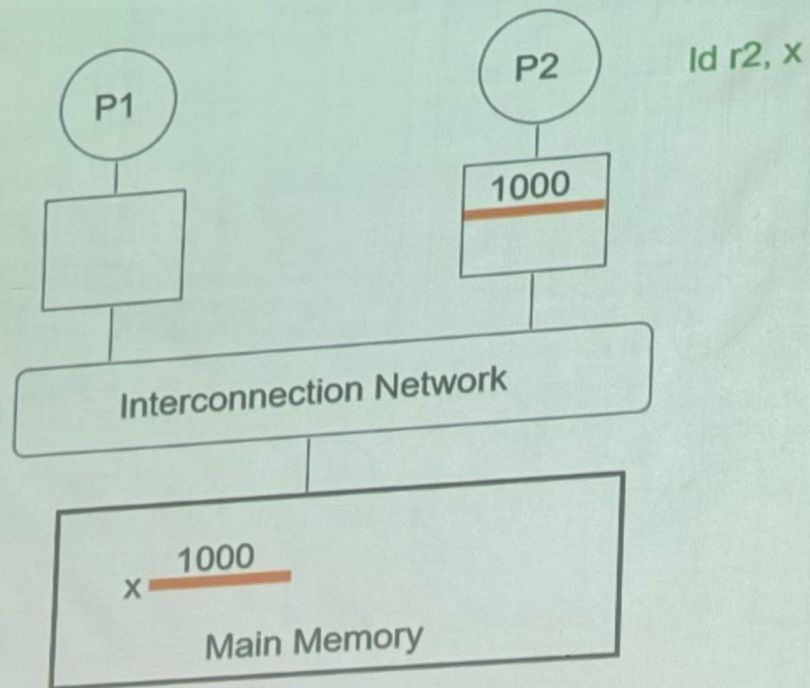
# Cache Coherence

- Basic question: If multiple processors cache the same block, how do they ensure they all see a consistent state?

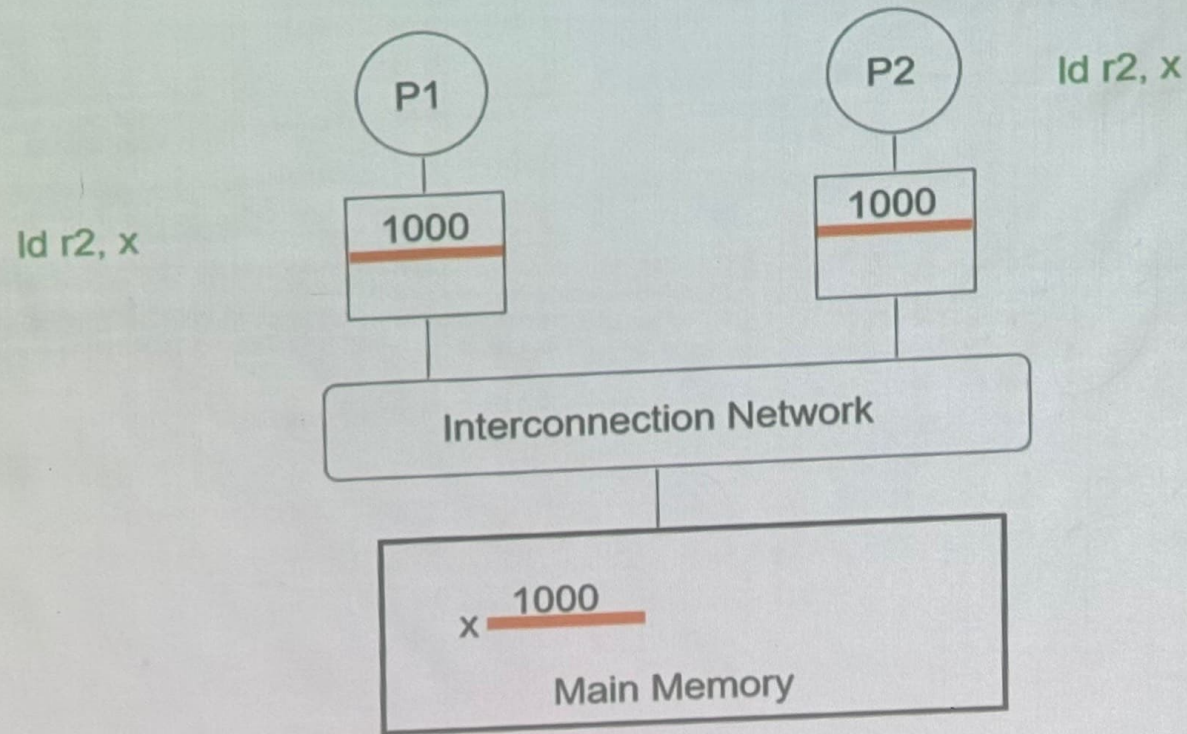




# The Cache Coherence Problem

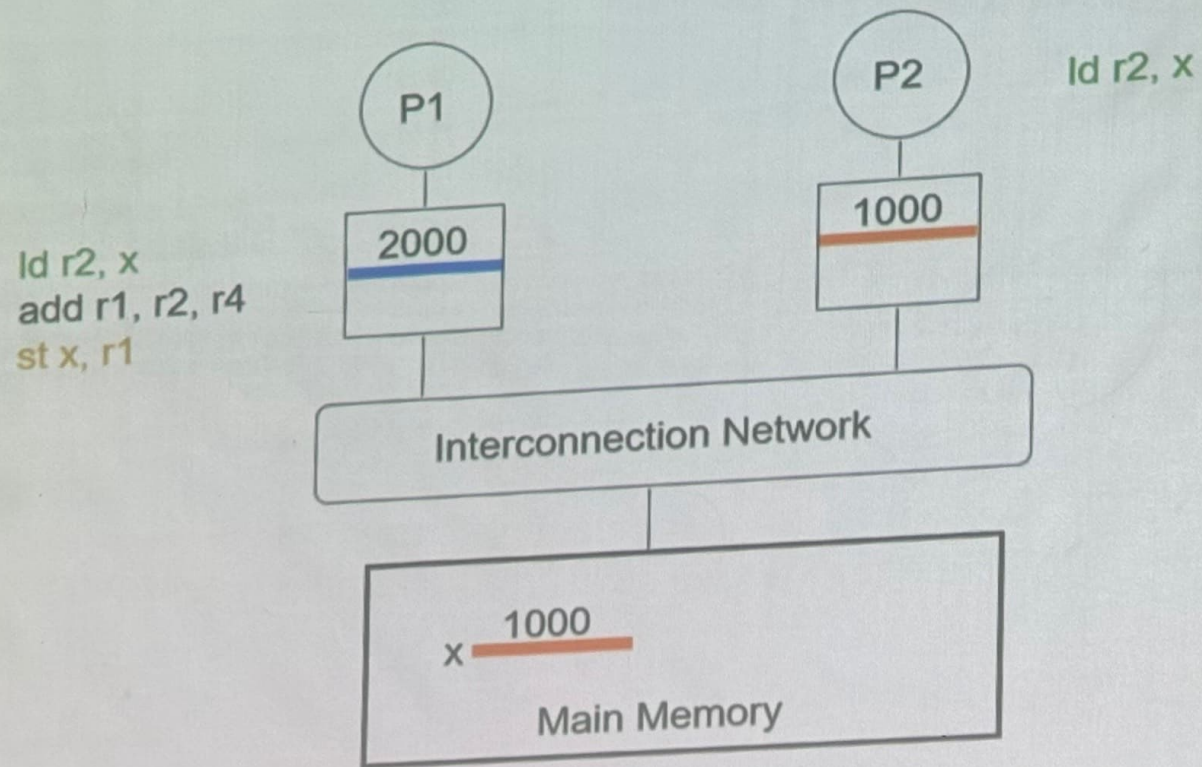


# The Cache Coherence Problem

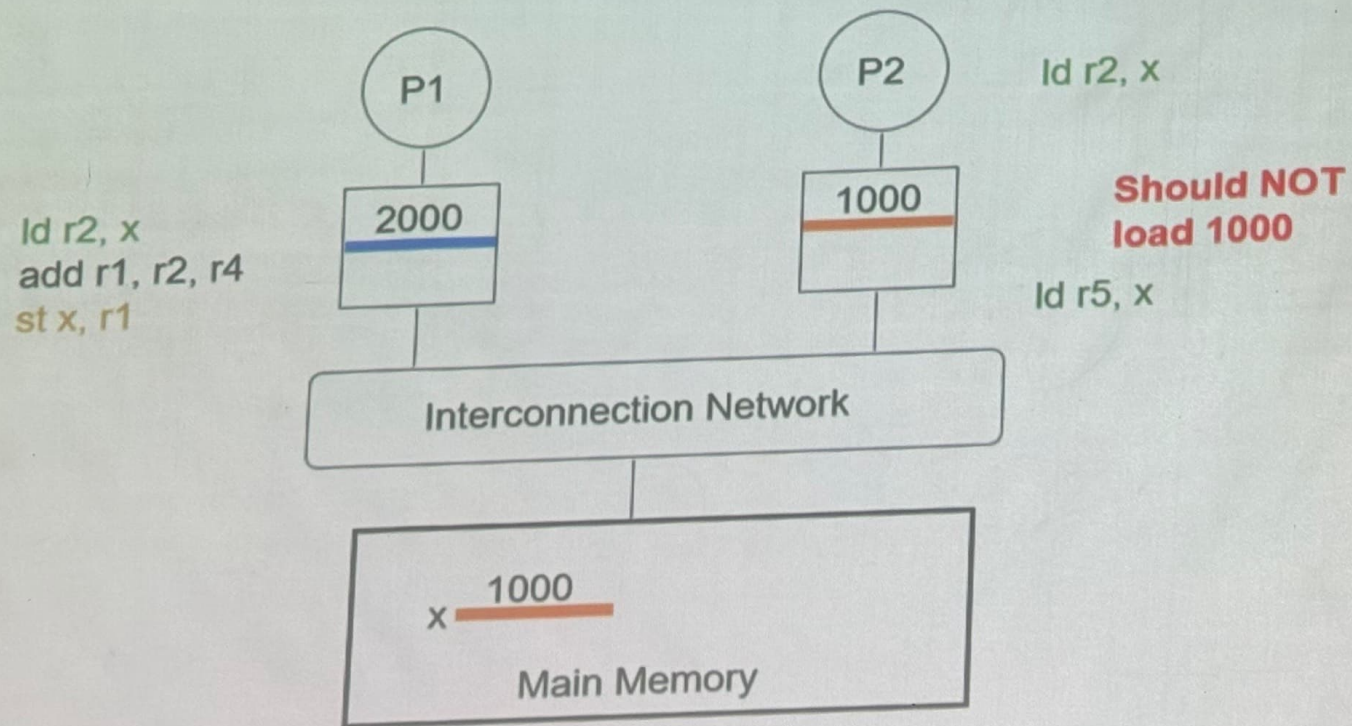




# The Cache Coherence Problem



# The Cache Coherence Problem



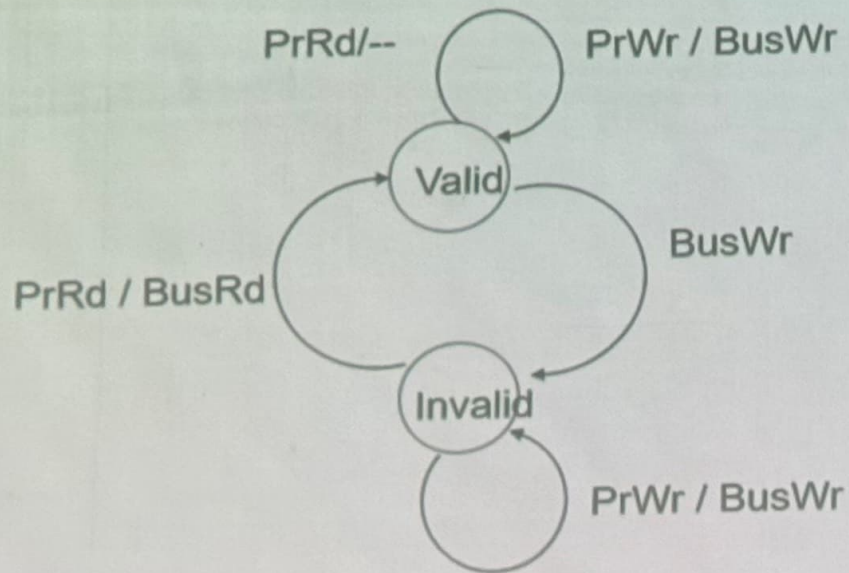


# Cache Coherence: Whose Responsibility?

- Software
  - Can programmer ensure coherence if caches invisible to software?
  - Coarse-grained: Page-level coherence has overheads
  - Non-solution: Make shared locks/data non-cacheable
  - A combination of non-cacheable and coarse-grained is doable
  - Fine-grained: What if the ISA provided a cache flush instruction?
    - FLUSH-LOCAL A: Flushes/invalidates the cache block containing address A from a processor's local cache.
    - FLUSH-GLOBAL A: Flushes/invalidates the cache block containing address A from all other processors' caches.
    - FLUSH-CACHE X: Flushes/invalidates all blocks in cache X.
- Hardware
  - Greatly simplifies software's job
  - One idea: Invalidate all other copies of block A when a core writes to A

## *A Very Simple Coherence Scheme (VI)*

- Caches “snoop” (observe) each other’s write/read operations. If a processor writes to a block, all others invalidate the block.
- A simple protocol:

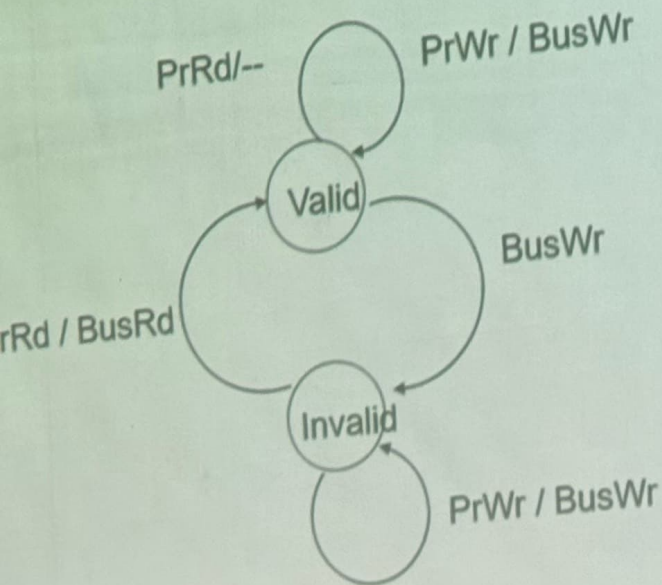




**Very Simple Consistency**

noop" (observe) each other's write/read operations. If a processor writes a block, all others invalidate the block.

protocol:



- Write-through, no-write-allocate cache
- Actions of the local processor on the cache block: PrRd, PrWr,
- Actions that are broadcast on the bus for the block: BusRd, BusWr

# *(Non-)Solutions to Cache Coherence*

- No hardware based coherence
  - Keeping caches coherent is software's responsibility
  - + Makes microarchitect's life easier
  - Makes average programmer's life much harder
    - need to worry about hardware caches to maintain program correctness?
  - Overhead in ensuring coherence in software (e.g., page protection, page-based software coherence, non-cacheable)
- All caches are shared between all processors
  - + No need for coherence
  - Shared cache becomes the bandwidth bottleneck
  - Very hard to design a scalable system with low-latency cache access this way



# Maintaining Coherence

- Need to guarantee that all processors see a consistent value (i.e., consistent updates) for the same memory location
- Writes to location A by P0 should be seen by P1 (eventually), and all writes to A should appear in some order
- Coherence needs to provide:
  - **Write propagation:** guarantee that updates will propagate
  - **Write serialization:** provide a consistent order seen by all processors for the same memory location
- Need a global point of serialization for this store ordering

## *Coherence: Update vs. Invalidate*

- How can we safely update replicated data?
  - Option 1 (Update protocol): push an update to all copies
  - Option 2 (Invalidate protocol): ensure there is only one copy (local), update it
- **On a Read:**
  - If local copy is Invalid, put out request



## *Coherence: Update vs. Invalidate (II)*

- **On a Write:**

- Read block into cache as before

### **Update Protocol:**

- Write to block, and simultaneously broadcast written data and address to sharers
- (Other nodes update the data in their caches if block is present)

### **Invalidate Protocol:**

- Write to block, and simultaneously broadcast invalidation of address to sharers
- (Other nodes invalidate block in their caches if block is present)



# *Update vs. Invalidate Tradeoffs*

- Which do we want?
  - Write frequency and sharing behavior are critical
- **Update**
  - + If sharer set is constant and updates are infrequent, avoids the cost of invalidate-reacquire (broadcast update pattern)
  - If data is rewritten without intervening reads by other cores, updates would be useless
  - Write-through cache policy → bus becomes bottleneck
- **Invalidate**
  - + After invalidation broadcast, core has exclusive access rights
  - + Only cores that keep reading after each write retain a copy
  - If write contention is high, leads to ping-ponging (rapid invalidation-reacquire traffic from different processors)



## Two Cache Coherence Methods

- How do we ensure that the proper caches are updated?
- **Snoopy Bus** [Goodman ISCA 1983, Papamarcos+ ISCA 1984]
  - Bus-based, single point of serialization *for all memory requests*
  - Processors observe other processors' actions
    - E.g.: P1 makes "read-exclusive" request for A on bus, P0 sees this and invalidates its own copy of A
- **Directory** [Censier and Feautrier, IEEE ToC 1978]
  - Single point of serialization *per block*, distributed among nodes
  - Processors make explicit requests for blocks
  - Directory tracks which caches have each block
  - Directory coordinates invalidation and updates
    - E.g.: P1 asks directory for exclusive copy, directory asks P0 to invalidate, waits for ACK, then responds to P1

# *Snoopy Cache Coherence*



# Snoopy

- Idea:

- All caches “snoop” all other caches’ read/write requests and keep the cache block coherent
- Each cache block has “coherence metadata” associated with it in the tag store of each cache

- Easy to implement if all caches share a common bus
  - Each cache broadcasts its read/write operations on the bus
  - Good for small-scale multiprocessors

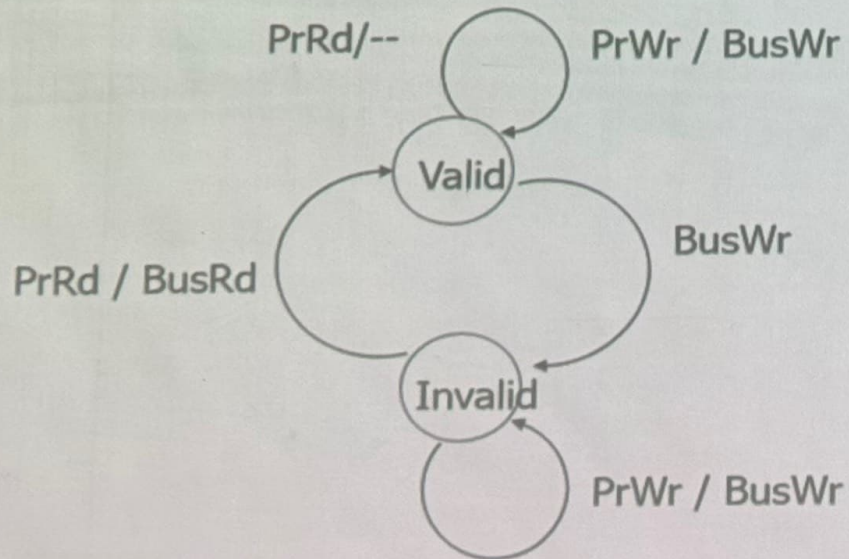
# Snoopy

- Idea:
  - All caches "snoop" all other caches' read/write requests and keep the cache block coherent
  - Each cache block has "coherence metadata" associated with it in the tag store of each cache
- Easy to implement if all caches share a common bus
  - Each cache broadcasts its read/write operations on the bus
  - Good for small-scale multiprocessors
  - What if you would like to have a 10,000-node multiprocessor?



## *A Simple Snoopy Cache Coherence Protocol*

- Caches “snoop” (observe) each others’ write/read operations
- A simple protocol (VI protocol):



- **Write-through, no-write-allocate** cache
- Actions of the local processor on the cache block: PrRd, PrWr,
- Actions that are broadcast on the bus for the block: BusRd, BusWr

6-oct

# Caveats of parallelism

Amdahl's law

(slide)

$$S + P = 1$$

Seq. ~~bad~~ bottleneck

$$\text{Max Speedup} = 1/f \quad (\text{theoretically})$$

Gustafson's law

More to think about —

(slide)

→

$$P = 64$$

$$\alpha = 0.05 \quad \text{or} \quad 5\%$$

$$\frac{S + P}{S + P/n}$$

$$= 1 / (S + (1 - S)/n)$$

$S_A$   
↓

$$1/0.05$$

$$= 20$$

X

$$\approx 15$$

$S_q$

$$\frac{1}{20 + \frac{19}{1}}$$

$$S_q \approx 60$$

Q2

$$P = 8$$

$$\alpha = 20\%$$

$$S_A = 3.3$$

$$S_q = 6.7$$



## Linear and Super linear Speedup

### Superlinear speedup

Can you speedup  $> N$  for  $N$  processor

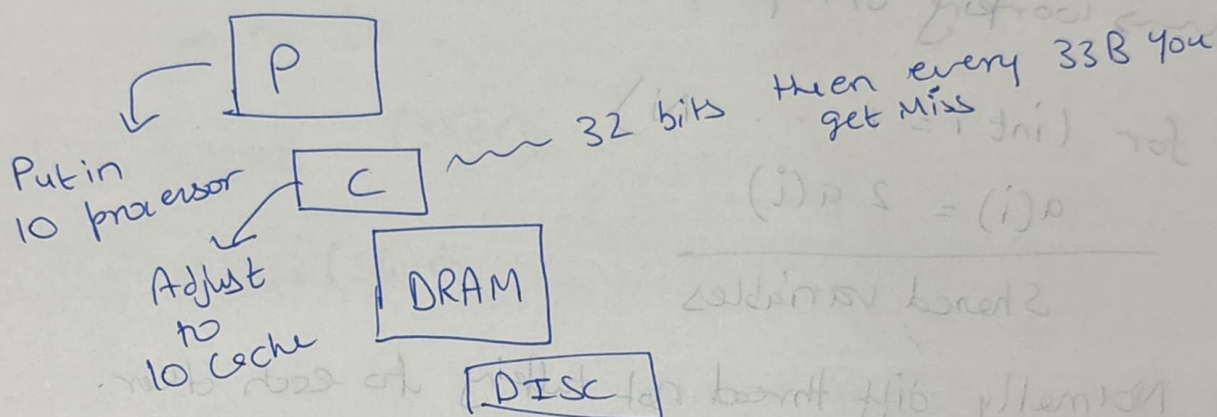
↳ in principle - No

↳ in reality  $\rightarrow$  Yes

if you bring

Searching in <sup>(billion)</sup> 10<sup>9</sup> integers

↳ Serial time . 80 KB



## Branch and bound ?

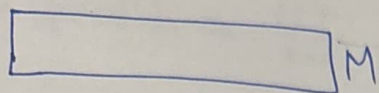
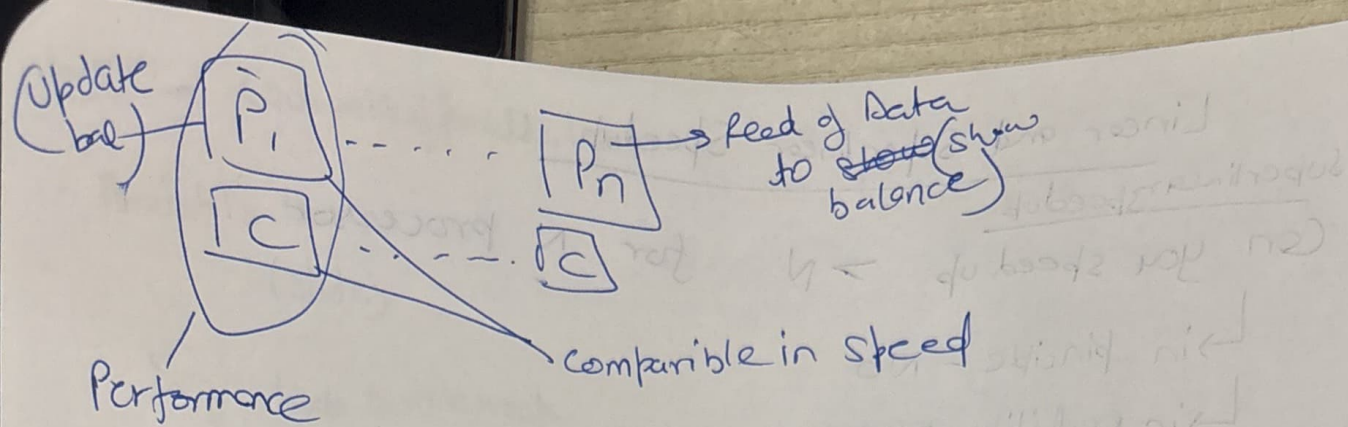
↳ How Do you solve travelling salesman problem?

## Bottleneck in parallel

- Synchronization

- Load imbalance (Slide)

- Resource contention



in parallel Architecture it becomes problem

→ working on parallel workloads

for (int  $i = 1 \dots N$ )

$$a(i) = 2 a(i)$$

Shared variables

Normally diff thread not talking to each other

let  $P_i$  - update bal

$P_k$  - show bal

Problem - You may update your balance or someone else balance

- other person might be giving his bal.

which value should  $P_k$  see  
the value in cache or updated value of  $P_i$

depends on where the cache network level is  
Can be in Disc, DRAM, Cache, etc.



# Consistency vs Coherence in Memory

↑  
entire  
memory

↑  
single  
location

How you maintain coherence

Suppose  $P_1, P_2$  Read  $x$

One of them write

then other one has to invalidate  
(System concept)

Invalidate All other Copies of block  $x$  when any  
write to it

## Cache Snoop (slide)

→ Non Solutions (slide)

memory consistency model  $\left\{ \begin{array}{l} \text{Sequential} \\ \text{Eventual} \end{array} \right\}$  will talk later

## Cache coherency model

- Snoopy bus

- Directory [After consistency models]

Read about

USB - Universal Serial Bus

Diff in 3.0 and 2.0  
Bus speed

It also can transmit power  
along with data

Laptop use HDMI

HDMI req power / Can ~~con~~ give  
power?

~~PCIe~~ PCIe can you have

- Interconnection N/w? - USB inside processor